CHAPTER 4

Neural Networks

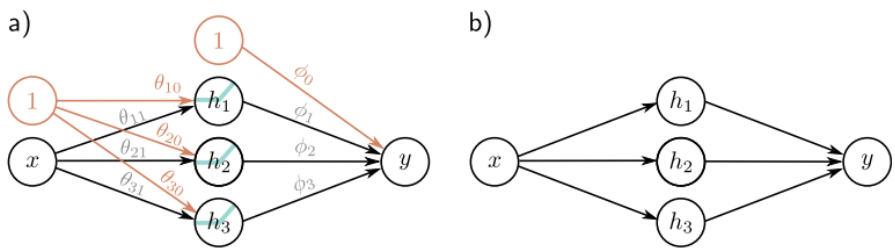
4.1. Shallow neural networks

Shallow neural networks are functions of the form $y = f[x, \{\theta, \phi\}]$, where ϕ and θ represent the network parameters, that map multivariate inputs x to multivariate outputs y.

4.1.1. Single input and output

For example, consider a network $f[x, \{\theta, \phi\}]$ that maps a scalar input x to a scalar output y and has ten parameters $\{\phi_0, \phi_1, \phi_2, \phi_3, \theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}, \theta_{30}, \theta_{31}\}$, and has the form:

$$y = f[x, \{\phi, \theta\}] = \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x]$$



We can divide this computation into three steps:

1. **Linear combination:** Compute three linear functions of the input x:

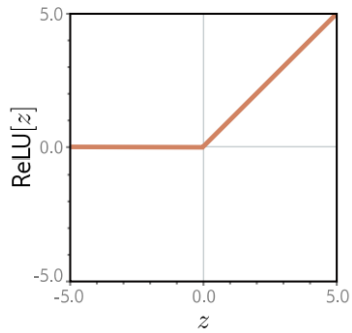
$$\theta_{10} + \theta_{11}x, \qquad \theta_{20} + \theta_{21}x, \qquad \theta_{30} + \theta_{31}x$$

2. **Activation:** Apply an activation function $a[\cdot]$ to each of these linear outputs.
3. **Weighted sum:** Multiply the three activations by the weights ϕ_1, ϕ_2, ϕ_3, sum them, and add the bias term ϕ_0 to produce the final output y.

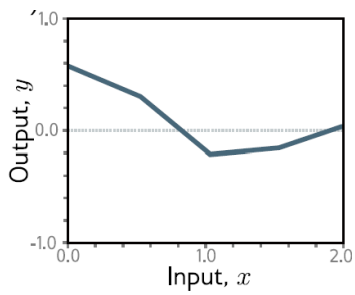
To complete the description, we need to specify the activation function $a[\cdot]$. There are many options, but the most widely used is the rectified linear unit (ReLU):

$$a[z] = ReLU[z] = \begin{cases} 0 & z < 0 \\ z & z \geq 0 \end{cases}$$

It outputs the input value if it is positive, and zero otherwise



The equation $y = f[x, \{\phi, \theta\}] = \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x]$ represents a family of continuous piecewise linear functions.



To simplify the explanation, we divide the function into two parts. First, we define the intermediate quantities:

$$h_1 = a[\theta_{10} + \theta_{11}x]$$
$$h_2 = a[\theta_{20} + \theta_{21}x]$$
$$h_3 = a[\theta_{30} + \theta_{31}x]$$

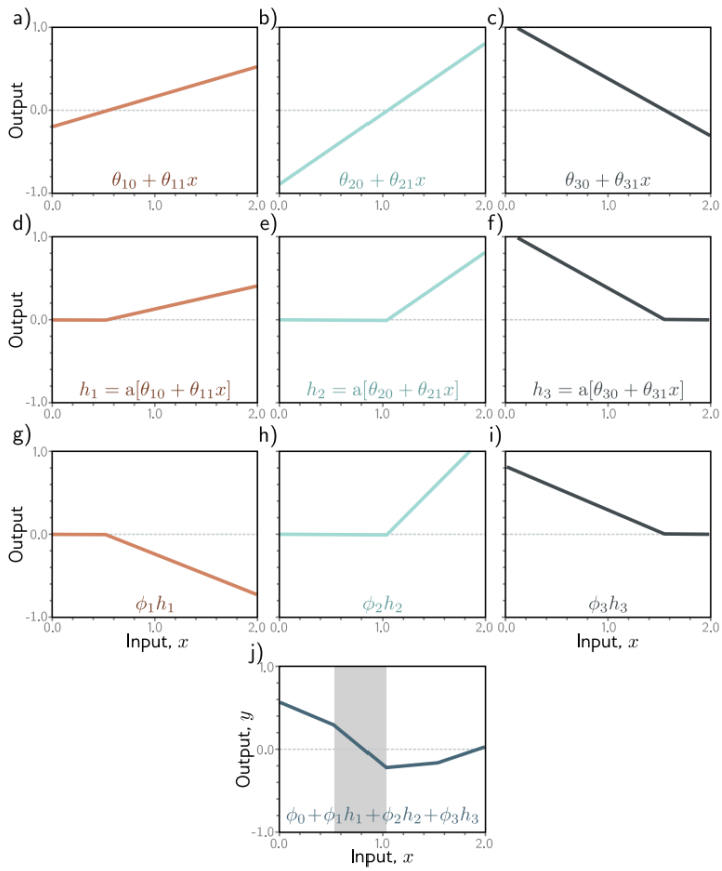
Here, h_1, h_2, and h_3 are referred to as hidden units. Next, the output is computed by combining these hidden units through a linear function:

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

Each hidden unit applies a linear function of the input, $\theta_{.0} + \theta_{.1}x$, which is then clipped at zero by the ReLU activation $a[\cdot]$. The points where these linear functions cross zero become the "joints" in the final output. The resulting clipped lines are then multiplied by the weights ϕ_1, ϕ_2, and ϕ_3, and a bias ϕ_0 is added to control the overall vertical position of the function.

Each linear segment corresponds to a different activation pattern of the hidden units. A hidden unit is active when its output is above zero and inactive when it is clipped. For example, in the shaded region, only h_1 and h_3 are active and contribute to the output, while h_2 is inactive. The slope of each linear segment is determined by the original slopes $\theta_{.1}$ of the active hidden units multiplied by their corresponding weights $\phi_{.}$. In the shaded region, the slope is $\theta_{11}\phi_1 + \theta_{31}\phi_3$, where the first term comes from h_1 and the second from h_3.

Each hidden unit contributes one "joint" to the function. With three hidden units, the function can have up to four linear regions. However, only three of these slopes are independent; the fourth slope is either zero (if all hidden units are inactive) or a combination of slopes from the other regions.



4.1.2. Multivariate inputs and outputs

In the previous example, the network takes a single scalar input x and produces a single scalar output y. This framework can be generalized to handle multivariate cases, where the network maps an input vector $x = [x_1, x_2, \dots, x_{D_i}]^T$ to an output vector $y = [y_1, y_2, \dots, y_{D_o}]^T$. We begin by examining how the model can be extended to predict multivariate outputs, then we address the case of multivariate inputs. Finally, we provide a general definition of a shallow neural network.

4.1.2.1. Multivariate outputs

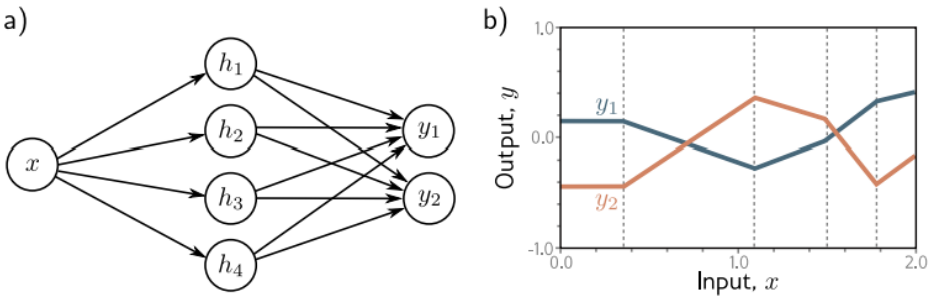
To generalize the network for multivariate outputs y, we assign a separate linear function of the hidden units to each output. For example, a network with a scalar input x, four hidden units h_1, h_2, h_3, h_4, and a two-dimensional output $y = [y_1, y_2]^T$ can be defined as follows:

$$h_1 = a[\theta_{10} + \theta_{11}x]$$
$$h_2 = a[\theta_{20} + \theta_{21}x]$$
$$h_3 = a[\theta_{30} + \theta_{31}x]$$
$$h_4 = a[\theta_{40} + \theta_{41}x]$$

And

$$y_1 = \phi_{10} + \phi_{11}h_1 + \phi_{12}h_2 + \phi_{13}h_3 + \phi_{14}h_4$$
$$y_2 = \phi_{20} + \phi_{21}h_1 + \phi_{22}h_2 + \phi_{23}h_3 + \phi_{24}h_4$$

The "joints" in the piecewise linear functions are determined by where the initial linear combinations $\theta_0 + \theta_1 x$ are clipped by the ReLU activations $a[\cdot]$ at the hidden units. Since both outputs y_1 and y_2 are linear functions of the same four hidden units, their four "joints" occur at the same input locations. However, the slopes of the linear segments and the overall vertical offsets can differ between the two outputs.



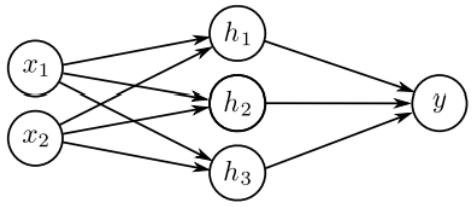
4.1.2.2. Multivariate inputs

To handle multivariate inputs x, we generalize the linear relationships between the inputs and the hidden units. For instance, a network with two inputs $x = [x_1, x_2]^T$ and a scalar output y could have three hidden units defined as follows:

$$h_1 = a[\theta_{10} + \theta_{11}x_1 + \theta_{12}x_2]$$
$$h_2 = a[\theta_{20} + \theta_{21}x_1 + \theta_{22}x_2]$$
$$h_3 = a[\theta_{30} + \theta_{31}x_1 + \theta_{32}x_2]$$

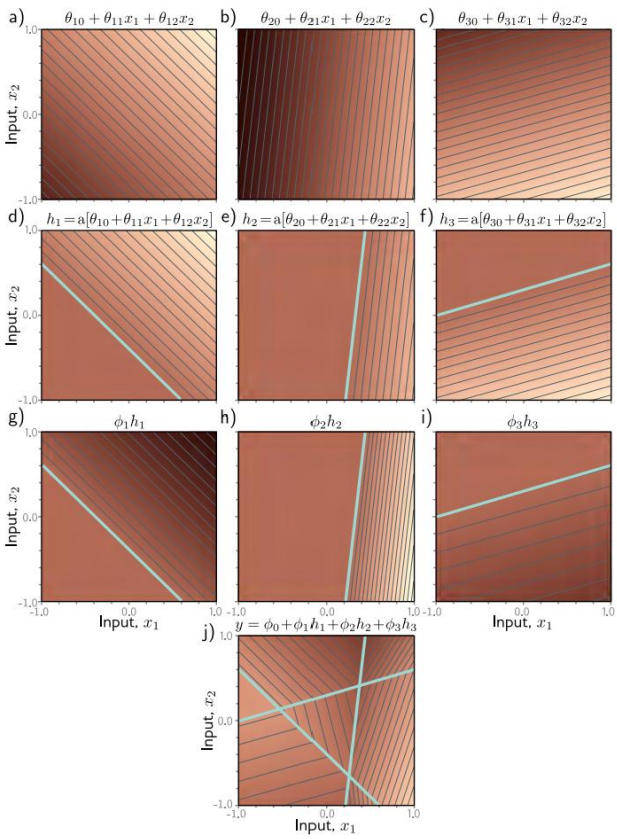
Here, each input has its own slope parameter. The hidden units are then combined in the usual manner to produce the output:

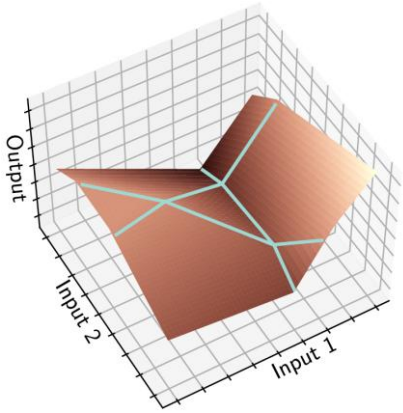
$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$



Each hidden unit computes a linear combination of the two inputs, forming an oriented plane in the 3D input-output space. The activation function then clips any negative values of these planes to zero. These clipped planes are combined through a second linear function to produce a continuous piecewise linear surface made up of convex polygonal regions, where each region corresponds to a distinct pattern of hidden-unit activations. For example, in the central triangular region, the first and third hidden units are active, while the second is inactive.

With more than two inputs, visualization becomes challenging, but the interpretation remains the same. The output remains a continuous piecewise linear function of the inputs, with the linear regions now forming convex polytopes in the higher-dimensional input space.





4.1.3. Shallow neural networks: general case

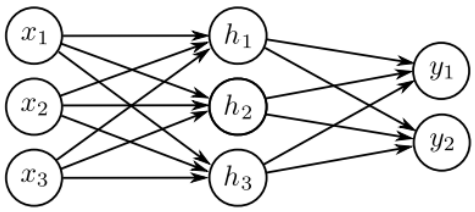
We now provide a general formulation for a shallow neural network $y = f[x, \phi]$, which maps a multi-dimensional input $x \in \mathbb{R}^{D_i}$ to a multi-dimensional output $y \in \mathbb{R}^{D_o}$ using $h \in \mathbb{R}^{D}$ hidden units. Each hidden unit is computed as follows:

$$h_d = a[\theta_d + \sum_{i=1}^{D_i} \theta_{di} x_i]$$

These hidden units are then combined linearly to produce the output:

$$y_i = \phi_j + \sum_{d=1}^{D} \phi_{jd} h_d$$

The model's parameters are given by $\{\theta_{..}, \phi_{..}\}$. The following figure illustrates an example with three inputs, three hidden units, and two outputs.



4.1.4. Activation functions

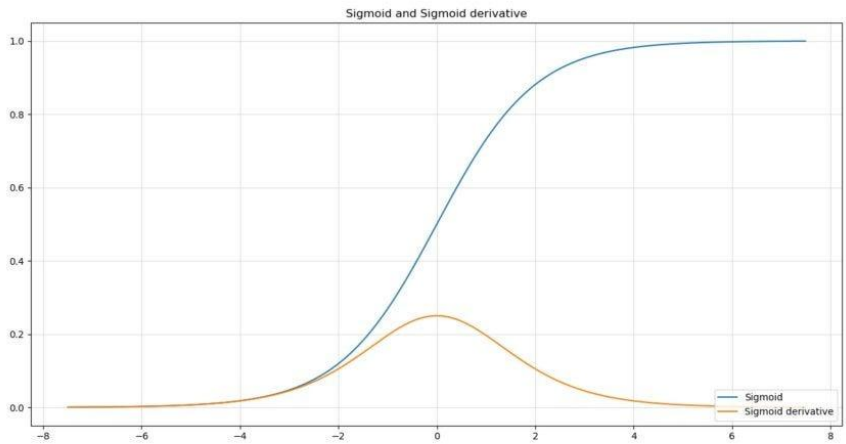
The activation function enables the model to capture nonlinear relationships between inputs and outputs, so it must itself be nonlinear. Without an activation, or with a linear one, the network could only represent linear mappings. Many activation functions have been explored, but the most common is ReLU, which is also easy to interpret. With ReLU activations, the network partitions the input space into convex polytopes formed by the intersections of hyperplanes at the "joints" of the ReLU functions. Each polytope corresponds to a distinct linear function. While the polytopes are the same for all outputs, the linear functions they contain can differ.

4.1.4.1. Sigmoid

The Sigmoid activation function is one of the classical activation functions and remains among the most widely recognized, aside from ReLU. It maps inputs $x \in [-\infty, +\infty]$ to outputs $y \in [0,1]$, with the steepest variation occurring in the interval $x \in [-4, +4]$.

However, using Sigmoid can introduce two significant limitations in a machine learning model. First, its outputs are not centered around zero; for example, when $x = 0$, the output is $y = 0.5$. This lack of symmetry can slow convergence during optimization. Second, the maximum value of its derivative is approximately 0.25 (at $x = 0$). As gradients are propagated backward through multiple layers, they tend to shrink rapidly. In deep networks, this leads to the vanishing gradient problem, which can slow down learning considerably or even prevent effective training.

$$f(x) = \frac{1}{1 + e^{-x}}$$

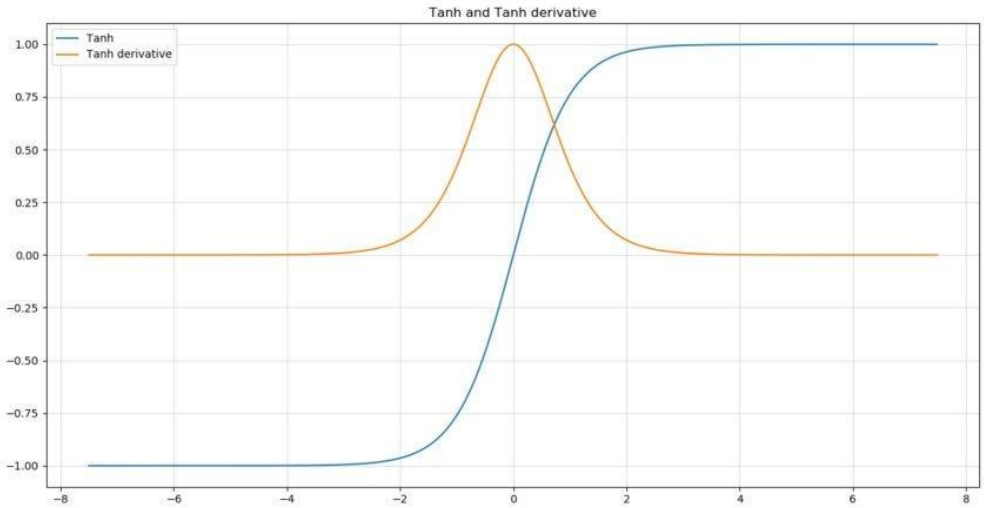


4.1.4.2. Tanh

Another widely used activation function is the hyperbolic tangent, or Tanh. It maps inputs from the entire real line to the range $y \in [-1, +1]$.

Although Tanh is zero-centered and therefore symmetric around the origin, it still suffers from the vanishing gradient problem. To address this limitation, alternative activation functions were later introduced.

$$f(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$$



4.1.4.3. Variants of ReLU (Leaky ReLU, PReLU, ELU)

The standard Rectified Linear Unit (ReLU) is widely used and often effective, but its tendency to output zero for all negative inputs can lead to the "dying ReLU" problem. If a neuron's weights cause its input to remain negative, the neuron will always produce zero. Consequently, its gradient during backpropagation is also zero, preventing any further weight updates and effectively rendering the neuron inactive.

To overcome this limitation, several ReLU variants have been proposed. These maintain the standard ReLU behavior for positive inputs while allowing a small, nonzero slope for negative inputs, ensuring that gradients can continue to flow even when the neuron is weakly activated.

a) **Leaky ReLU:** is a simple modification of the standard ReLU. Rather than outputting zero for negative inputs, it produces a small negative value proportional to the input. Formally, it is defined as:

$$f(x) = \begin{cases} x & if\ x > 0 \\ \alpha x & if\ x \leq 0 \end{cases}$$

Here, α is a small, fixed hyperparameter, typically around 0.01. By providing a nonzero gradient for negative inputs, the Leaky ReLU prevents neurons from "dying" and ensures they can keep learning.

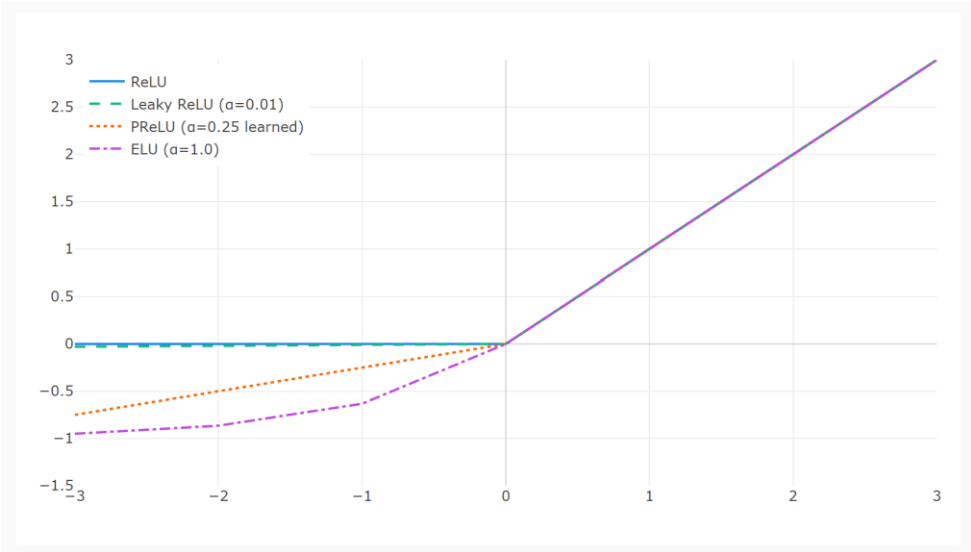
b) **Parametric ReLU (PReLU):** Parametric ReLU (PReLU) extends the idea of the Leaky ReLU by making the negative slope α a learnable parameter rather than a fixed value. This allows the network to determine the optimal slope for negative inputs during training via backpropagation.

$$f(x) = \begin{cases} x & if\ x > 0 \\ \alpha x & if\ x \leq 0 \quad (\alpha\ is\ learned) \end{cases}$$

PReLU can outperform Leaky ReLU when the optimal negative slope differs from a fixed small value. However, it adds a learnable parameter for each neuron (or channel in convolutional layers), which can slightly increase the risk of overfitting, especially when data is limited.

c) **Exponential Linear Unit (ELU):** is another ReLU variant designed to mitigate the dying ReLU problem. Like Leaky ReLU, it allows negative inputs, but instead of a fixed slope, ELU uses a smooth, continuously differentiable curve for negative values. The steepness of this negative region is controlled by a hyperparameter α.

$$f(x) = \begin{cases} x & if\ x > 0 \\ \alpha(e^x - 1) & if\ x \leq 0 \end{cases}$$



4.2. Deep neural networks

4.2.1. Composing neural networks

To better understand the behavior of deep neural networks, we begin by examining the composition of two shallow networks, where the output of the first serves as the input to the second. Consider two shallow networks, each containing three hidden units. The first network receives an input x and produces an output y, and is defined as follows:

$$h_1 = a[\theta_{10} + \theta_{11}x]$$
$$h_2 = a[\theta_{20} + \theta_{21}x]$$
$$h_3 = a[\theta_{30} + \theta_{31}x]$$

And

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

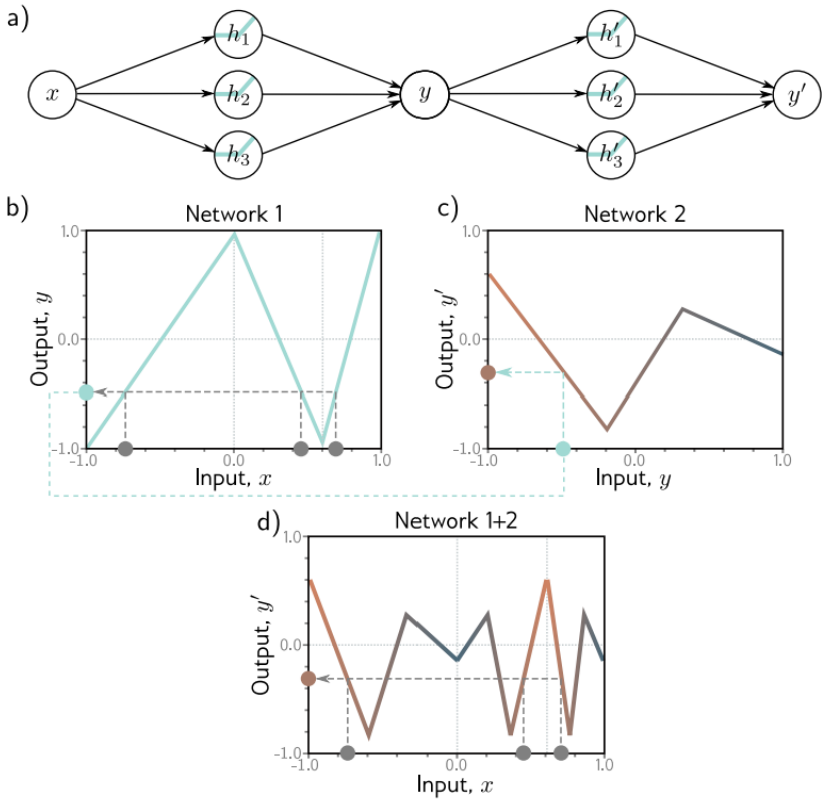
The second network takes y as input and returns y' and is defined by:

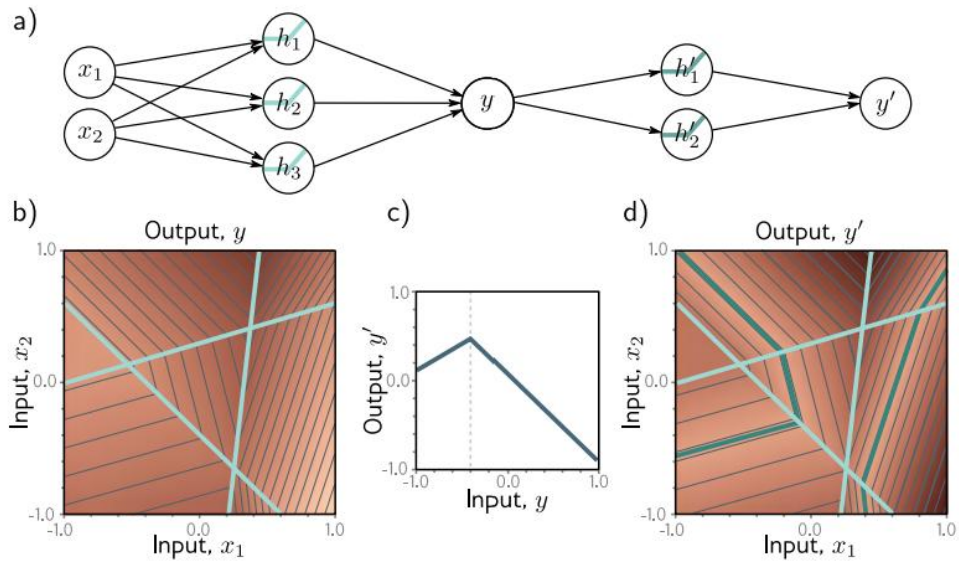
$$h'_1 = a[\theta'_{10} + \theta'_{11}x]$$
$$h'_2 = a[\theta'_{20} + \theta'_{21}x]$$
$$h'_3 = a[\theta'_{30} + \theta'_{31}x]$$

And

$$y' = \phi'_0 + \phi'_1 h'_1 + \phi'_2 h'_2 + \phi'_3 h'_3$$

With ReLU activations, this model still represents a family of piecewise linear functions. However, it can produce more linear regions than a shallow network containing six hidden units. To understand why, suppose the first network generates three alternating segments with positive and negative slopes. In this case, three distinct intervals of x are mapped to the same output range $y \in [-1,1]$, and the second network then applies its transformation to this range three separate times. As a result, the function implemented by the second network is effectively replicated three times, yielding nine linear regions in total. The same idea extends naturally to higher-dimensional inputs.





4.2.2. From composing networks to deep networks

The previous section demonstrated that complex functions can be constructed by feeding the output of one shallow neural network into a second network. We now show that this configuration is in fact a special case of a deep network with two hidden layers.

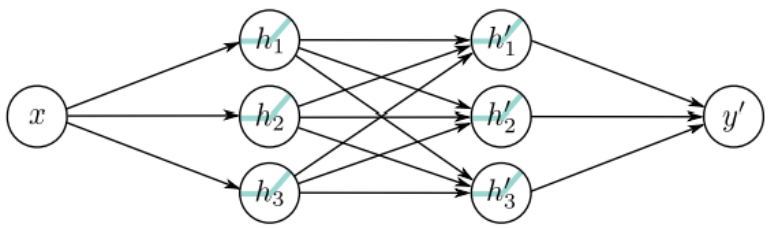
The output of the first network, $y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$, is a linear combination of its hidden unit activations. In the second network, the initial computations—such as $\theta'_{10} + \theta'_{11} y$, $\theta'_{20} + \theta'_{21} y$, and $\theta'_{30} + \theta'_{31} y$—are linear functions of this output. Since the composition of two linear functions remains linear, applying one linear transformation to another results in a new linear function.

$$h'_1 = a[\theta'_{10} + \theta'_{11} y] = a[\theta'_{10} + \theta'_{11}\phi_0 + \theta'_{11}\phi_1 h_1 + \theta'_{11}\phi_2 h_2 + \theta'_{11}\phi_3 h_3]$$
$$h'_2 = a[\theta'_{20} + \theta'_{21} y] = a[\theta'_{20} + \theta'_{21}\phi_0 + \theta'_{21}\phi_1 h_1 + \theta'_{21}\phi_2 h_2 + \theta'_{21}\phi_3 h_3]$$
$$h'_3 = a[\theta'_{30} + \theta'_{31} y] = a[\theta'_{30} + \theta'_{31}\phi_0 + \theta'_{31}\phi_1 h_1 + \theta'_{31}\phi_2 h_2 + \theta'_{31}\phi_3 h_3]$$

which we can rewrite as:

$$h'_1 = a[\psi_{10} + \psi_{11} h_1 + \psi_{12} h_2 + \psi_{13} h_3]$$
$$h'_2 = a[\psi_{20} + \psi_{21} h_1 + \psi_{22} h_2 + \psi_{23} h_3]$$
$$h'_3 = a[\psi_{30} + \psi_{31} h_1 + \psi_{32} h_2 + \psi_{33} h_3]$$

Where $\psi_{.0} = \theta'_{.0} + \theta'_{.1}\phi_0$, $\psi_{.1} = \theta'_{.1}\phi_1$, $\psi_{.2} = \theta'_{.1}\phi_2$, and $\psi_{.3} = \theta'_{.1}\phi_3$. The result is a network with two hidden layers.



4.2.3. Deep neural networks

In the previous section, we demonstrated that combining two shallow networks results in a specific instance of a deep network with two hidden layers. We now move to the more general formulation of a deep network with two hidden layers, each composed of three hidden units. The first hidden layer is defined as follows:

$$h_1 = a[\theta_{10} + \theta_{11} x]$$
$$h_2 = a[\theta_{20} + \theta_{21} x]$$
$$h_3 = a[\theta_{30} + \theta_{31} x]$$

the second layer by:

$$h'_1 = a[\psi_{10} + \psi_{11} h_1 + \psi_{12} h_2 + \psi_{13} h_3]$$
$$h'_2 = a[\psi_{20} + \psi_{21} h_1 + \psi_{22} h_2 + \psi_{23} h_3]$$
$$h'_3 = a[\psi_{30} + \psi_{31} h_1 + \psi_{32} h_2 + \psi_{33} h_3]$$

and the output by:

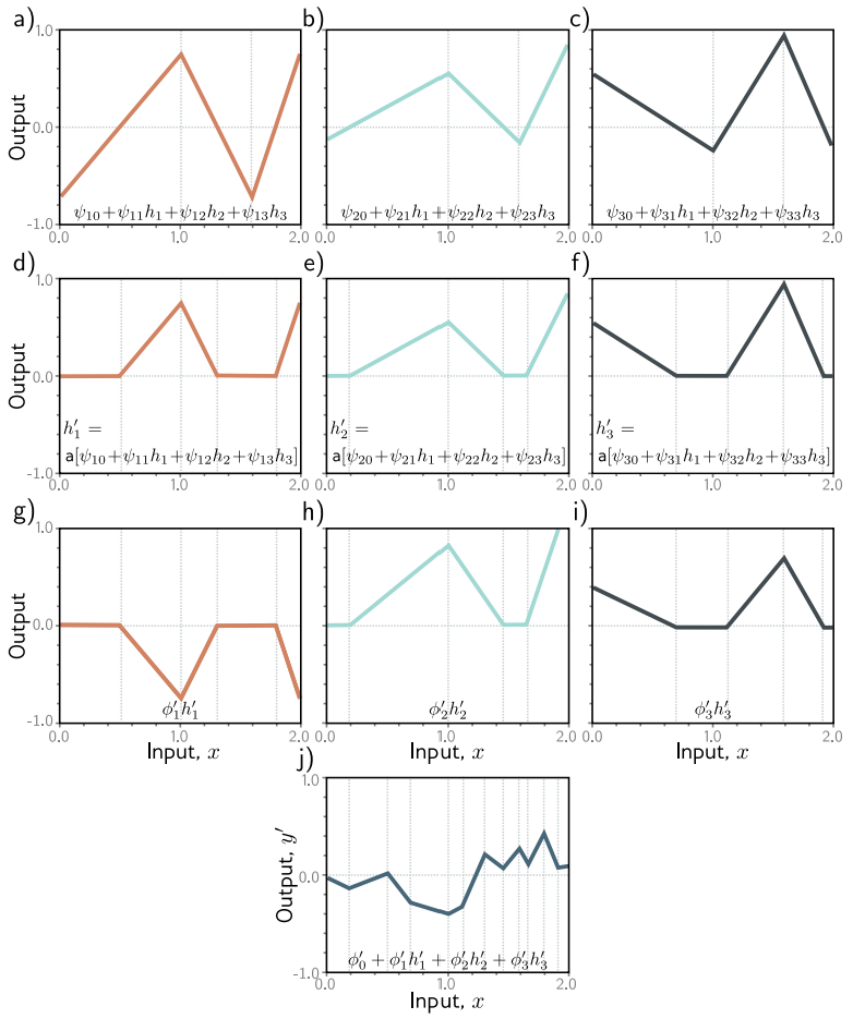
$$y' = \phi'_0 + \phi'_1 h'_1 + \phi'_2 h'_2 + \phi'_3 h'_3$$

Examining these equations provides another perspective on how the network progressively builds a more complex function:

1. The three hidden units $h_1, h_2,$ and h_3 in the first layer are computed in the usual way: linear combinations of the input are formed and then passed through ReLU activation functions.
2. The pre-activations in the second layer are obtained by taking new linear combinations of these

hidden units. At this stage, the model can be viewed as a shallow network with three outputs, producing three piecewise linear functions whose "joints" occur at the same input locations.

3. Each of these functions is then passed through another ReLU activation at the second hidden layer, which clips the functions and introduces additional "joints."
4. Finally, the network output is computed as a linear combination of the activations in the second hidden layer.



4.2.3.1. Hyperparameters

The construction of deep networks can be extended beyond two hidden layers; modern architectures may contain hundreds of layers, each with thousands of hidden units. The number of hidden units in a layer is called the width of the network, while the number of hidden layers defines its depth. The total number of hidden units provides a measure of the network's capacity.

We denote the number of layers by K and the number of hidden units in each layer by $D_1, D_2, \dots, D_K$. These quantities are examples of hyperparameters, meaning they are chosen prior to learning the model parameters (such as weights and biases). For a fixed set of hyperparameters—for example, $K = 2$ layers with $D_k = 3$ hidden units per layer—the network defines a family of functions, and the learned parameters specify a particular function within that family. When hyperparameters are also taken into account, neural networks can therefore be viewed as representing a *family of families* of functions that map inputs to outputs.

4.2.3.2. Matrix notation

We have seen that a deep neural network is composed of successive linear transformations interleaved with activation functions. The equations presented above can alternatively be expressed in matrix form as follows:

$$\begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} = a\left(\begin{bmatrix} \theta_{10} \\ \theta_{20} \\ \theta_{30} \end{bmatrix} + \begin{bmatrix} \theta_{11} \\ \theta_{21} \\ \theta_{31} \end{bmatrix} x\right)$$

$$\begin{bmatrix} h'_1 \\ h'_2 \\ h'_3 \end{bmatrix} = a\left(\begin{bmatrix} \psi_{10} \\ \psi_{20} \\ \psi_{30} \end{bmatrix} + \begin{bmatrix} \psi_{11} & \psi_{12} & \psi_{13} \\ \psi_{21} & \psi_{22} & \psi_{23} \\ \psi_{31} & \psi_{32} & \psi_{33} \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix}\right)$$

$$y' = \phi'_0 + [\phi'_1 \quad \phi'_2 \quad \phi'_3] \begin{bmatrix} h'_1 \\ h'_2 \\ h'_3 \end{bmatrix}$$

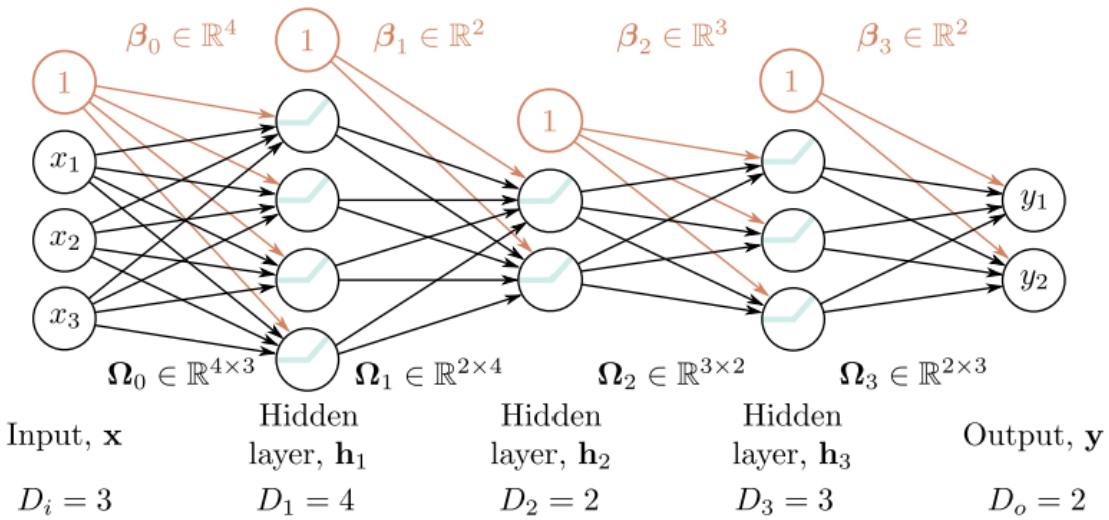
General case

This matrix notation can become cumbersome for networks with many layers. To simplify, we will denote the vector of hidden units at layer k as h_k, the vector of biases contributing to layer $k+1$ as β_k, and the weights applied to layer $k$that feed into layer $k+1$ as Ω_k. Using this notation, a general deep network with K layers can be written as:

$$h_1 = a[\beta_0 + \Omega_0 x]$$
$$h_2 = a[\beta_1 + \Omega_1 h_1]$$
$$h_3 = a[\beta_2 + \Omega_2 h_2]$$

$$h_K = a[\beta_{K-1} + \Omega_{K-1} h_{K-1}]$$
$$y = \beta_K + \Omega_K h_K$$

The parameters of this model consist of all the weight matrices and bias vectors $\{\beta_k, \Omega_k\}_{k=0}^{K}$. If the kth layer has D_k hidden units, the bias vector β_{k-1} has size D_k, while the final bias vector β_K has size D_o, matching the output dimension. The first weight matrix Ω_0 has dimensions $D_1 \times D_i$, where D_i is the input size, and the last weight matrix Ω_K has dimensions $D_o \times D_K$. All intermediate weight matrices Ω_k have dimensions $D_{k+1} \times D_k$. Using this notation, the entire network can equivalently be expressed as a single function:



4.3. Gradient descent

To fit a model, we require a training dataset $\{x_i, y_i\}$ composed of input–output pairs. The objective is to determine the model parameters ϕ such that the function $f(x_i, \phi)$ maps the inputs x_i to the corresponding outputs y_i as accurately as possible. For this purpose, we define a loss function $L(\phi)$ that produces a scalar value measuring the discrepancy between the predicted outputs and the true outputs. An optimization algorithm is then used to find the parameter values $\hat{\phi}$ that minimize this loss:

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}}\ L[\phi]$$

There are many classes of optimization algorithms, but the methods most commonly used to train neural networks are iterative. These approaches start by initializing the model parameters using heuristic rules and then repeatedly update them in a manner that progressively reduces the loss.

The simplest algorithm in this class is gradient descent. It begins with an initial set of parameters $\phi = [\phi_0, \phi_1, \dots, \phi_N]^T$ and proceeds iteratively through two steps:

Step 1. Compute the derivatives of the loss function with respect to the parameters.

Step 2. Update the parameters using the rule

$$\phi \leftarrow \phi - \alpha \nabla_\phi L(\phi)$$

where the positive scalar α controls the size of the update.

In the first step, the gradient of the loss function at the current parameter values is calculated. This gradient indicates the direction in which the loss increases most rapidly. In the second step, the parameters are adjusted slightly in the opposite direction (hence the negative sign), which corresponds to moving downhill on the loss surface.

The parameter α can be fixed, commonly referred to as the learning rate, or determined through a line search, where several candidate values are tested to select the one that yields the greatest reduction in loss. At the minimum of the loss function, the surface becomes flat, meaning that further improvement is no longer possible. Consequently, the gradient approaches zero and the parameters stop changing. In practice, the optimization process is typically terminated when the magnitude of the gradient becomes sufficiently small.

4.3.1. Gradient descent in linear regression

Consider applying gradient descent to the one-dimensional linear regression model. In this case, the model $f(x, \phi)$ maps a scalar input x to a scalar output y. The parameter vector is $\phi = [\phi_0, \phi_1]^T$, where ϕ_0 corresponds to the intercept and ϕ_1 represents the slope of the line:

$$y = f[x, \phi] = \phi_0 + \phi_1 x$$

Given a dataset $\{x_i, y_i\}$ consisting of *n* input–output pairs, we adopt the least squares loss function:

$$L[\phi] = \sum_{i=1}^{n} \ell_i = \sum_{i=1}^{n} (f[x_i, \phi] - y_i)^2 = \sum_{i=1}^{n} (\phi_0 + \phi_1 x_i - y_i)^2$$

Here, the term $\ell_i = (\phi_0 + \phi_1 x_i - y_i)^2$ represents the contribution of the ith training example to the total loss. The derivative of the overall loss function with respect to the parameters can therefore be expressed as the sum of the derivatives of these individual contributions.

$$\frac{\partial L}{\partial \phi} = \frac{\partial}{\partial \phi} \sum_{i=1}^{n} \ell_i = \sum_{i=1}^{n} \frac{\partial \ell_i}{\partial \phi}$$

These derivatives are given by:

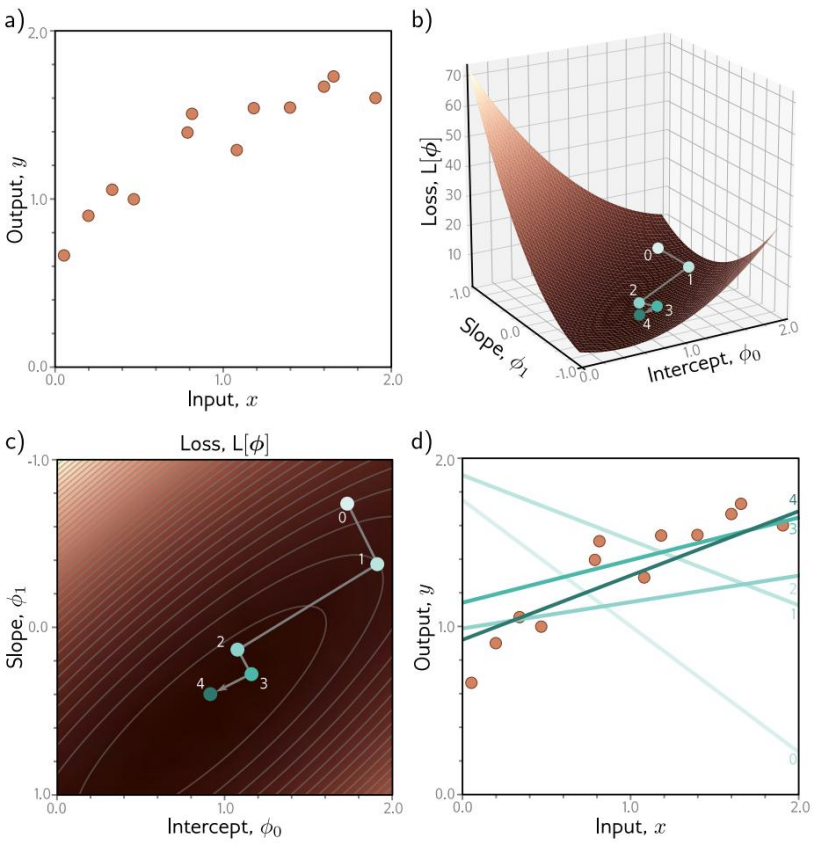
$$\frac{\partial \ell_i}{\partial \phi} = \begin{bmatrix} \frac{\partial \ell_i}{\partial \phi_0} \\ \frac{\partial \ell_i}{\partial \phi_1} \end{bmatrix} = \begin{bmatrix} 2(\phi_0 + \phi_1 x_i - y_i) \\ 2x_i(\phi_0 + \phi_1 x_i - y_i) \end{bmatrix}$$

The parameters ϕ_0 and ϕ_1 are updated as follows:

$$\phi_0 \leftarrow \phi_0 - \alpha \sum_{i=1}^{n} 2(\phi_0 + \phi_1 x_i - y_i)$$

$$\phi_1 \leftarrow \phi_1 - \alpha \sum_{i=1}^{n} 2x_i(\phi_0 + \phi_1 x_i - y_i)$$

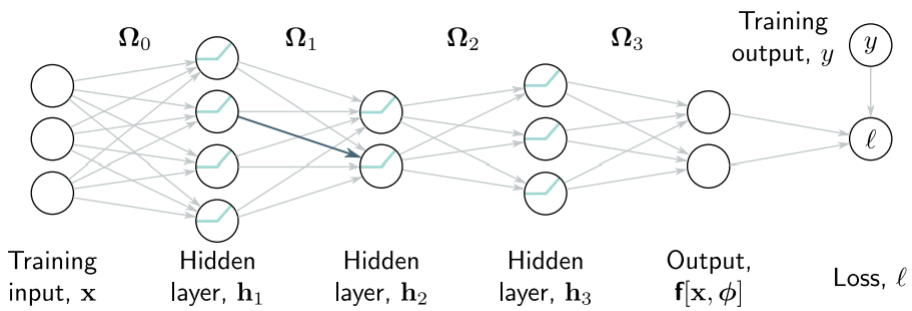
The figure bellow illustrates the progression of this algorithm as the derivatives are repeatedly computed using the equations above and the parameters are subsequently updated. In this example, a line search strategy is employed at each iteration to determine the value of α that produces the greatest reduction in the loss.



4.3.2. Gradient descent in deep neural network

Consider a neural network $f(x, \phi)$ with a multivariate input x, the set of model parameters is $\phi = \{\beta_0, \Omega_0, \beta_1, \Omega_1, \beta_2, \Omega_2, \beta_3, \Omega_3\}$, and three hidden layers denoted by h_1, h_2, and h_3:

$$h_1 = a[\beta_0 + \Omega_0 x]$$
$$h_2 = a[\beta_1 + \Omega_1 h_1]$$
$$h_3 = a[\beta_2 + \Omega_2 h_2]$$
$$y = \beta_3 + \Omega_3 h_3$$



We also define individual loss terms ℓ_i, which measure the negative log-likelihood of the true label y_i given the model prediction $f(x_i, \phi)$. The total loss is then obtained by summing these terms over all training examples:

$$L[\phi] = \sum_{i=1}^{n} \ell_i$$

The most widely used algorithm for training neural networks is stochastic gradient descent (SGD), which updates the parameters according to:

$$\phi_{t+1} \leftarrow \phi_t - \alpha \sum_{i \in B_t} \frac{\partial \ell_i[\phi_t]}{\partial \phi}$$

Here, α is the learning rate, and B_t denotes the set of batch indices at iteration t. To perform this update, we compute the derivatives $\frac{\partial \ell_i}{\partial \beta_k}$ and $\frac{\partial \ell_i}{\partial \Omega_k}$ for the parameters $\{\beta_k, \Omega_k\}$ at every layer $k \in \{0,1, \dots, K\}$ and for each example i in the batch.

4.3.3. Backpropagation algorithm

We aim to apply the backpropagation algorithm to this network. In this setting, the intermediate variables f_k and h_k are vectors, the biases β_k are vectors, the weights Ω_k are matrices, and the activation functions are ReLUs.

Forward pass: The network computations can be expressed as a sequence of successive operations:

$$f_0 = \beta_0 + \Omega_0 x_i$$
$$h_1 = a[f_0]$$
$$f_1 = \beta_1 + \Omega_1 h_1$$
$$h_2 = a[f_1]$$
$$f_2 = \beta_2 + \Omega_2 h_2$$
$$h_3 = a[f_2]$$
$$f_3 = \beta_3 + \Omega_3 h_3$$
$$\ell_i = l[f_3, y_i]$$

Here, f_{k-1} denotes the pre-activation values at the kth hidden layer (i.e., the inputs to the ReLU function $\mathrm{a}(\cdot)$), while $\mathrm{h_k}$ contains the post-activation values of the kth hidden layer (i.e., the outputs after applying ReLU). The term $\ell(f_3, y_i)$ represents the loss function, such as least squares or binary cross-entropy. During the forward pass, we perform these computations sequentially and store all intermediate values for use in backpropagation.

Backward pass #1: Next, we examine how the loss varies with changes in the pre-activation vectors f_0, f_1, f_2. Using the chain rule, the derivative of the loss ℓ_i with respect to f_2 can be expressed as:

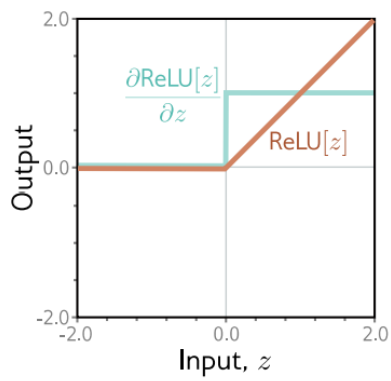
$$\frac{\partial \ell_i}{\partial f_2} = \frac{\partial h_3}{\partial f_2}\frac{\partial f_3}{\partial h_3}\frac{\partial \ell_i}{\partial f_3}$$

The three factors on the right-hand side have dimensions $D_3 \times n$, $D_3 \times D_o$, and $D_o \times n$, where D_3 is the number of hidden units in the third layer and D_o is the dimensionality of the model output f_3.
By propagating backward through this expression, we obtain:

- The derivative $\frac{\partial \ell_i}{\partial f_3}$ of the loss ℓ_i with respect to the network output f_3 depends on the chosen loss function, but it typically has a simple form.
- The derivative $\frac{\partial f_3}{\partial h_3}$ of the network output with respect to the hidden layer h_3 is given by:

$$\frac{\partial f_3}{\partial h_3} = \frac{\partial}{\partial h_3}(\beta_3 + \Omega_3 h_3) = \Omega_3^T$$

- The derivative $\frac{\partial h_3}{\partial f_2}$ of the activated output h_3 with respect to its input f_2 depends on the activation function. For ReLU activations, the derivative entries are 0 wherever $f_2 < 0$ and 1 elsewhere, represented as a vector $I[f_2 > 0]$.



Hence:

$$\frac{\partial \ell_i}{\partial f_2} = I[f_2 > 0] \odot \left(\Omega_3^T \frac{\partial \ell_i}{\partial f_3}\right)$$

Similarly, we can determine how the loss varies with respect to f_1 and f_0:

$$\frac{\partial \ell_i}{\partial f_1} = \frac{\partial h_2}{\partial f_1}\frac{\partial f_2}{\partial h_2}\left(\frac{\partial h_3}{\partial f_2}\frac{\partial f_3}{\partial h_3}\frac{\partial \ell_i}{\partial f_3}\right) = I[f_1 > 0] \odot \left(\Omega_2^T \frac{\partial \ell_i}{\partial f_2}\right)$$

$$\frac{\partial \ell_i}{\partial f_0} = \frac{\partial h_1}{\partial f_0}\frac{\partial f_1}{\partial h_1}\left(\frac{\partial h_2}{\partial f_1}\frac{\partial f_2}{\partial h_2}\frac{\partial h_3}{\partial f_2}\frac{\partial f_3}{\partial h_3}\frac{\partial \ell_i}{\partial f_3}\right) = I[f_0 > 0] \odot \left(\Omega_1^T \frac{\partial \ell_i}{\partial f_1}\right)$$

Notice that in each case, the bracketed term was already computed in the preceding step. By propagating backward through the network, these computations can be reused, making the process more efficient. Additionally, each term is straightforward to calculate.

Backward pass #2: Having computed $\frac{\partial \ell_i}{\partial f_k}$, we can now focus on the derivatives of the loss with respect to the weights and biases. The derivatives with respect to the bias vectors β_k are obtained using the chain rule:

$$\frac{\partial \ell_i}{\partial \beta_k} = \frac{\partial f_k}{\partial \beta_k}\frac{\partial \ell_i}{\partial f_k} = \frac{\partial}{\partial \beta_k}(\beta_k + \Omega_k h_k)\frac{\partial \ell_i}{\partial f_k} = \frac{\partial \ell_i}{\partial f_k}$$

Similarly, the derivative with respect to the weight matrix Ω_k is given by:

$$\frac{\partial \ell_i}{\partial \Omega_k} = \frac{\partial f_k}{\partial \Omega_k}\frac{\partial \ell_i}{\partial f_k} = \frac{\partial}{\partial \Omega_k}(\beta_k + \Omega_k h_k)\frac{\partial \ell_i}{\partial f_k} = \frac{\partial \ell_i}{\partial f_k} h_k^T$$

4.3.4. Backpropagation algorithm summary

We now provide a brief summary of the backpropagation algorithm. Consider a deep neural network $f(x_i, \phi)$ that takes input x_i, contains K hidden layers with ReLU activations, and has an individual loss term $\ell_i = \ell[f(x_i, \phi), y_i]$. The objective of backpropagation is to compute the derivatives of the loss with respect to the biases β_k and weights Ω_k, namely $\frac{\partial \ell_i}{\partial \beta_k}$ and $\frac{\partial \ell_i}{\partial \Omega_k}$.

Forward pass: Compute and store the following intermediate values:

$$f_0 = \beta_0 + \Omega_0 x_i$$
$$h_k = a[f_{k-1}] \qquad k \in \{1,2,\ldots,K\}$$
$$f_k = \beta_k + \Omega_k h_k \qquad k \in \{1,2,\ldots,K\}$$

Backward pass: Begin with the derivative $\frac{\partial \ell_i}{\partial f_K}$ of the loss ℓ_i with respect to the network output f_K and propagate it backward through the network:

$$\frac{\partial \ell_i}{\partial \beta_k} = \frac{\partial \ell_i}{\partial f_k} \qquad k \in \{K, K-1, \ldots, 1\}$$

$$\frac{\partial \ell_i}{\partial \Omega_k} = \frac{\partial \ell_i}{\partial f_k} h_k^T \qquad k \in \{K, K-1, \ldots, 1\}$$

$$\frac{\partial \ell_i}{\partial f_{k-1}} = I[f_{k-1} > 0] \odot \left(\Omega_k^T \frac{\partial \ell_i}{\partial f_k}\right) \qquad k \in \{K, K-1, \ldots, 1\}$$

Here, $\odot$ represents elementwise (pointwise) multiplication, and $I[f_{k-1} > 0]$ is a vector with entries equal to 1 where $f_{k-1} > 0$ and 0 elsewhere. Finally, we calculate the derivatives with respect to the first layer's biases and weights:

$$\frac{\partial \ell_i}{\partial \beta_0} = \frac{\partial \ell_i}{\partial f_0}$$
$$\frac{\partial \ell_i}{\partial \Omega_0} = \frac{\partial \ell_i}{\partial f_0} x_i^T$$

These derivatives are computed for each training example in the batch and then summed to obtain the gradient used for the SGD parameter update.

The backpropagation algorithm is highly computationally efficient, as the most expensive operations in both the forward and backward passes are matrix multiplications (by Ω and Ω^T), which involve only additions and multiplications. However, it is memory intensive, because all intermediate values from the forward pass must be retained, which can constrain the size of the model that can be trained.

4.3.5. Parameter initialization

The backpropagation algorithm calculates the derivatives needed by stochastic gradient descent to update and train the model. Before training begins, it is essential to properly initialize the parameters. To understand why initialization is important, note that during the forward pass, each set of pre-activations f_k is computed as:

$$f_k = \beta_k + \Omega_k h_k = \beta_k + \Omega_k a[f_{k-1}]$$

Here, $a(\cdot)$ denotes the ReLU activation, and Ω_k and β_k are the weights and biases, respectively. Suppose we initialize all biases to zero and the elements of Ω_k from a normal distribution with mean

0 and variance σ^2. Two scenarios can arise:

- **Very small variance ($\sigma^2 \approx 10^{-5}$)**: Each element of $\beta_k + \Omega_k h_k$ is a weighted sum of h_k with tiny weights, so the resulting values tend to be smaller than the inputs. Additionally, the ReLU function zeroes out negative values, effectively halving the range of activations. As a result, the magnitudes of the pre-activations in successive hidden layers shrink progressively through the network.
- **Very large variance ($\sigma^2 \approx 10^5$)**: Each element of $\beta_k + \Omega_k h_k$ becomes a weighted sum with very large weights, producing outputs with much larger magnitudes than the inputs. Although ReLU halves the range of negative values, the pre-activations still grow larger as they propagate through the layers if σ^2 is sufficiently high.

In both cases, the pre-activation values can become so small or so large that they exceed the limits of finite-precision floating-point representation. The same issue occurs during the backward pass: each gradient update involves multiplication by Ω^T. Poorly chosen initialization can cause the gradient magnitudes to either shrink toward zero or blow up, leading to the vanishing gradient or exploding gradient problems. In the former, parameter updates become negligibly small, slowing or halting learning, while in the latter, updates become unstable and may prevent the network from converging.

4.3.5.1. Initialization for forward pass

We can formalize the previous argument mathematically. Consider the computation between two consecutive pre-activation vectors f_{k-1} and f_k with dimensions D_{k-1} and D_k, respectively:

$$f_k = \beta_k + \Omega_k a[f_{k-1}]$$

To maintain the variance of the pre-activations across layers, i.e., $\sigma^2_{f_k} = \sigma^2_{f_{k-1}}$, the weights should be initialized with variance:

$$\sigma^2_{\Omega_k} = \frac{2}{D_{k-1}}$$

where D_{k-1} is the number of units in the preceding layer. This initialization strategy is known as *He initialization*.

4.3.5.2. Initialization for backward pass

During backpropagation, the gradient with respect to the pre-activation, $\frac{\partial \ell}{\partial f_k}$, is propagated backward through the network. Each step involves multiplication by the transpose of the weight matrix, Ω^T:

$$\frac{\partial \ell_i}{\partial f_{k-1}} = I[f_{k-1} > 0] \odot \left(\Omega_k^T \frac{\partial \ell_i}{\partial f_k}\right)$$

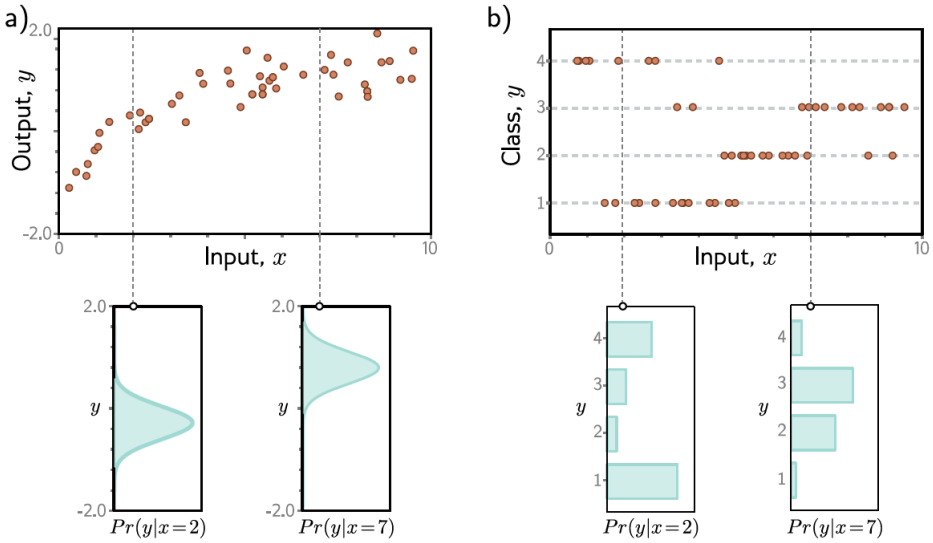
To avoid vanishing or exploding gradients, we aim to keep the variance of $\frac{\partial \ell_i}{\partial f_{k-1}}$ approximately equal to that of $\frac{\partial \ell_i}{\partial f_k}$. This can be ensured by initializing the weights so that:

$$\sigma^2_{\Omega_k} = \frac{2}{D_{k-1}}$$

where D_{k-1} is the number of units in the preceding layer (the same as in the forward pass). This ensures that both forward activations and backward gradients maintain a stable scale, avoiding vanishing or exploding values.

4.4. Loss function

Consider a model $f(x, \phi)$ with parameters ϕ that produces an output from an input x. Up to this point, we have assumed that the model directly outputs a prediction y. Here, we adopt a different viewpoint by interpreting the model as estimating a conditional probability distribution $Pr(y \mid x)$ over the possible outputs y given the input x. The loss function is then defined to encourage the model to assign a high probability to each training target y_i under the distribution $Pr(y_i \mid x_i)$ derived from its corresponding input x_i.



This change in perspective raises the question of how a model $f(x, \phi)$ can be adapted to produce a probability distribution. The approach is straightforward. First, we select a parametric probability distribution $Pr(y \mid \xi)$ defined over the output space y. Then, the network is used to estimate one or more parameters ξ of this distribution.

For instance, suppose the prediction domain consists of real numbers, so $y \in \mathbb{R}$. In this case, we may choose the univariate normal distribution, which is defined over $\mathbb{R}$. This distribution is characterized by its mean μ and variance σ^2, so $\xi = \{\mu, \sigma^2\}$. The machine learning model can be designed to predict the mean μ, while the variance σ^2 may be considered an unknown constant.

4.4.1. Maximum likelihood criterion

The model now produces a set of distribution parameters $\xi_i = f(x_i, \phi)$ for each training input x_i. Each observed target y_i should therefore have a high probability under its corresponding distribution $Pr(y_i \mid \xi_i)$. Consequently, the model parameters ϕ are chosen to maximize the joint probability of all n training samples:

$$\hat{\phi} = \underset{\phi}{\operatorname{argmax}} \left[\prod_{i=1}^{n} Pr(y_i | x_i) \right]$$

$$\hat{\phi} = \underset{\phi}{\operatorname{argmax}} \left[\prod_{i=1}^{n} Pr(y_i | \xi_i) \right]$$

$$\hat{\phi} = \underset{\phi}{\operatorname{argmax}} \left[\prod_{i=1}^{n} Pr(y_i | f[x_i, \phi]) \right]$$

The combined probability represents the likelihood of the model parameters; therefore, the objective above is referred to as the maximum likelihood criterion.

This formulation implicitly relies on two assumptions. First, the data are assumed to be identically distributed, meaning that the probability distribution governing the outputs y_i has the same form for every data point. Second, the conditional distributions $Pr(y_i \mid x_i)$ are assumed to be independent across samples. In other words, we assume that the data points are independent and identically distributed (i.i.d.). As a result, the overall likelihood of the training dataset can be expressed as the product of the individual conditional likelihoods:

$$Pr(y_1, y_2, \ldots, y_n | x_1, x_2, \ldots, x_n) = \prod_{i=1}^{n} Pr(y_i \mid x_i)$$

The maximum likelihood criterion is not always convenient in practice. Since each term $Pr(y_i \mid f(x_i, \phi))$ can be very small, multiplying many such terms may result in an extremely small value that is difficult to represent using finite-precision arithmetic. To address this issue, we instead maximize the logarithm of the likelihood, which is mathematically equivalent:

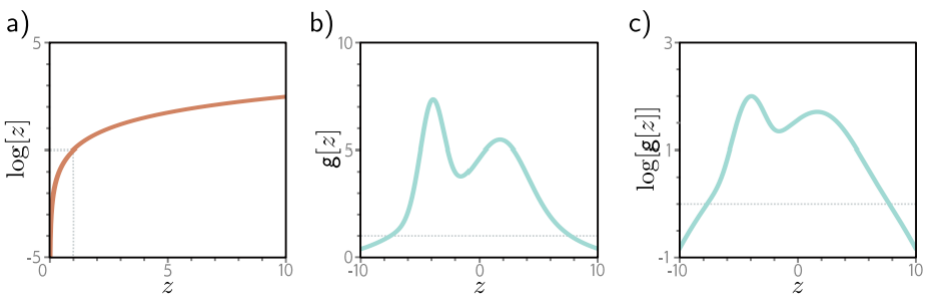
$$\hat{\phi} = \underset{\phi}{\operatorname{argmax}} \left[\prod_{i=1}^{n} Pr(y_i | f[x_i, \phi]) \right]$$

$$\hat{\phi} = \underset{\phi}{\operatorname{argmax}} \left[log \left[\prod_{i=1}^{n} Pr(y_i | f[x_i, \phi]) \right] \right]$$

$$\hat{\phi} = \underset{\phi}{\operatorname{argmax}} \left[\sum_{i=1}^{n} log[Pr(y_i | f[x_i, \phi])] \right]$$

The log-likelihood criterion is equivalent to the likelihood criterion because the logarithm is a monotonically increasing function: if $z > z'$, then $\log(z) > \log(z')$, and vice versa. Consequently, when the model parameters ϕ are adjusted to increase the log-likelihood, the original likelihood is also increased. This implies that both criteria reach their maximum at the same point, meaning that the optimal parameters $\hat{\phi}$ are identical in both formulations.

However, the log-likelihood offers a practical advantage: it converts the product of probabilities into a sum of terms, which avoids numerical issues associated with representing extremely small numbers in finite-precision arithmetic.



Finally, by convention, model fitting is typically formulated as a loss minimization problem. Therefore, to transform the maximum log-likelihood objective into a minimization task, we multiply it by -1. This leads to the negative log-likelihood (NLL) criterion, which is what forms the final loss function L[ϕ].

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}}\left[-\sum_{i=1}^{n} log[Pr(y_i|f[x_i,\phi])]\right]$$

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}}[L[\phi]]$$

4.4.2. Inference

The network no longer directly predicts the output y; instead, it estimates a probability distribution over possible values of y. During inference, however, we typically require a single predicted value rather than a full distribution. In such cases, we use the value that maximizes the predicted distribution as the point estimate.

$$\hat{y} = \underset{y}{\operatorname{argmax}}\left[Pr\left(y\middle|f[x,\hat{\phi}]\right)\right]$$

In many cases, this value can be expressed directly in terms of the distribution parameters θ predicted by the model. For instance, for a univariate normal distribution, the maximum of the distribution occurs at the mean μ.

4.4.3. Guidelines for Designing Loss Functions

The procedure for constructing loss functions for a training dataset $\{x_i, y_i\}$ using the maximum likelihood approach is as follows:

1. Select a probability distribution $Pr(y \mid \xi)$ over the output domain y, with parameters ξ.
2. Design the model $f(x,\phi)$ to predict one or more of these parameters, so that $\xi = f(x,\phi)$ and $Pr(y \mid \xi) = Pr(y \mid f(x,\phi))$.
3. Train the model by finding the parameters $\hat{\phi}$ that minimize the negative log-likelihood (NLL) loss over the training pairs $\{x_i, y_i\}$.

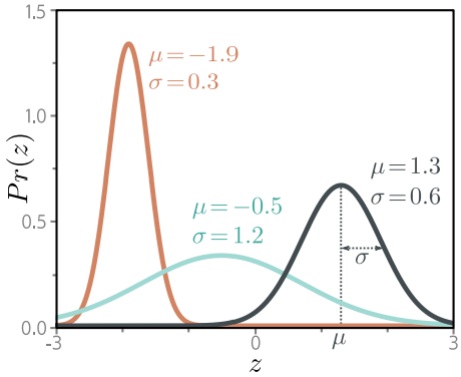
During inference for a new input x, return a point estimate corresponding to the distribution's maximum.

4.4.4. Examples

Example 1: univariate regression

We begin by examining univariate regression models, where the objective is to predict a single scalar output $y \in \mathbb{R}$ from an input $x$using a model $f(x,\phi)$ with parameters ϕ. Following the loss-function design procedure, we first choose a probability distribution over the output y. Here, we select the univariate normal distribution, which is defined over $y \in \mathbb{R}$. This distribution is characterized by two parameters: the mean μ and variance σ^2, and its probability density function is:

$$Pr(y|\mu,\sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} exp\left[-\frac{(y-\mu)^2}{2\sigma^2}\right]$$



Next, we configure the machine learning model $f(x,\phi)$ to predict one or more parameters of the chosen distribution. In this case, the model predicts only the mean, so $\mu = f(x,\phi)$:

$$Pr(y|f(x,\phi),\sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} exp\left[-\frac{(y-f(x,\phi))^2}{2\sigma^2}\right]$$

The goal is to find parameters ϕ that make the training data $\{x_i, y_i\}$ as probable as possible under this distribution. To achieve this, we define a loss function $L(\phi)$ based on the negative log-likelihood:

$$L[\phi] = -\sum_{i=1}^{n} log[Pr(y_i|f[x_i,\phi],\sigma^2)]$$

$$L[\phi] = -\sum_{i=1}^{n} log\left[\frac{1}{\sqrt{2\pi\sigma^2}} exp\left[-\frac{(y_i-f(x_i,\phi))^2}{2\sigma^2}\right]\right]$$

Training the model then involves finding the parameters $\hat{\phi}$ that minimize this loss.
Now, we proceed with some algebraic simplifications of the loss function. Our objective is to:

$$\hat{\phi} = \underset{\phi}{argmin}\left[-\sum_{i=1}^{n} log\left[\frac{1}{\sqrt{2\pi\sigma^2}} exp\left[-\frac{(y_i-f(x_i,\phi))^2}{2\sigma^2}\right]\right]\right]$$

$$\hat{\phi} = \underset{\phi}{argmin}\left[-\sum_{i=1}^{n} log\left[\frac{1}{\sqrt{2\pi\sigma^2}}\right] - \frac{(y_i-f(x_i,\phi))^2}{2\sigma^2}\right]$$

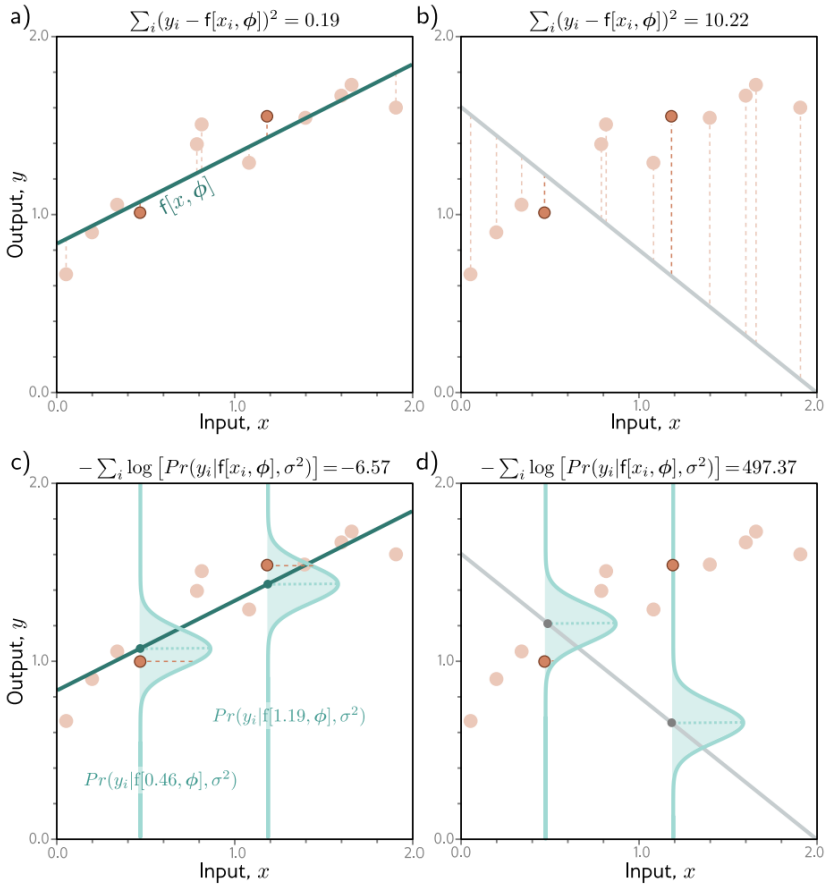
$$\hat{\phi} = \underset{\phi}{argmin}\left[-\sum_{i=1}^{n} - \frac{(y_i-f(x_i,\phi))^2}{2\sigma^2}\right]$$

$$\hat{\phi} = \underset{\phi}{argmin}\left[\sum_{i=1}^{n} (y_i-f(x_i,\phi))^2\right]$$

These manipulations lead to the least squares loss function, which we first encountered in the context of linear regression:

$$L[\phi] = \sum_{i=1}^{n} (y_i-f(x_i,\phi))^2$$

We observe that the least squares loss function arises naturally from the assumptions that the predictions are (i) independent and (ii) drawn from a normal distribution with mean $\mu = f(x_i,\phi)$.



The network no longer predicts y directly; instead, it predicts the mean $\mu = f(x, \phi)$ of a normal distribution over y. During inference, we typically require a single "best" estimate $\hat{y}$, which we obtain by taking the maximum of the predicted distribution.

For the univariate normal distribution, this maximum occurs at the mean μ. Since the model already predicts μ, the point estimate is simply $\hat{y} = f(x, \hat{\phi})$.

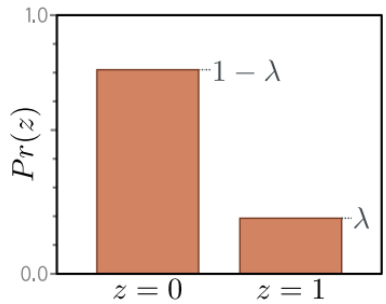
Example 2: binary classification

In binary classification, the task is to assign an input x to one of two discrete classes, $y \in \{0,1\}$. Here, y is referred to as a label. Following the procedure for designing loss functions, we first select a probability distribution over the output space $y \in \{0,1\}$. A natural choice is the Bernoulli distribution, which is defined on $\{0, 1\}$ and has a single parameter $\lambda \in [0,1]$ representing the probability that $y = 1$.

$$Pr(y|x) = \begin{cases} 1 - \lambda & y = 0 \\ \lambda & y = 1 \end{cases}$$

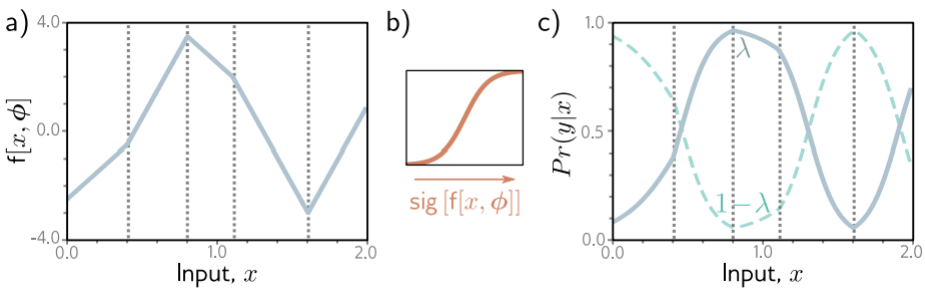
which can equivalently be written as:

$$Pr(y|x) = (1 - \lambda)^{1-y} . \lambda^y$$



Next, we configure the machine learning model $f(x,\phi)$ to predict the single distribution parameter λ. Since λ must lie within the range $[0, 1]$, but the network output can take any real value, we apply a function that maps $\mathbb{R}$ to $[0, 1]$. A common choice for this purpose is the logistic sigmoid function.

$$\lambda = \sigma(f(x,\phi)) = \frac{1}{1+e^{-[f(x,\phi)]}}$$



Thus, the predicted distribution parameter is given by $\lambda = \sigma[f(x,\phi)]$. The corresponding likelihood becomes:

$$Pr(y|x) = (1-\sigma[f(x,\phi)])^{1-y}.\sigma[f(x,\phi)]^{y}$$

The loss function is defined as the negative log-likelihood over the training set:

$$L[\phi] = -\sum_{i=1}^{n} log[Pr(y_i|f[x_i,\phi])]$$

$$L[\phi] = \sum_{i=1}^{n} -(1-y_i)log[1-\sigma[f(x_i,\phi)]] - y_i log[\sigma[f(x_i,\phi)]]$$

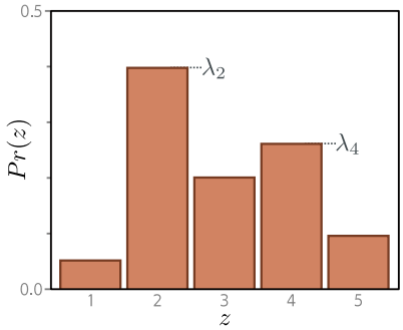
This loss is commonly referred to as the binary cross-entropy loss.

The transformed model output $\sigma[f(x,\phi)]$ predicts the Bernoulli parameter λ, representing the probability that $y=1$. Accordingly, $1-\lambda$ gives the probability that $y=0$. During inference, if a single point estimate of y is needed, we assign $y=1$ when $\lambda > 0.5$ and $y=0$ otherwise.

Example 3: multiclass classification

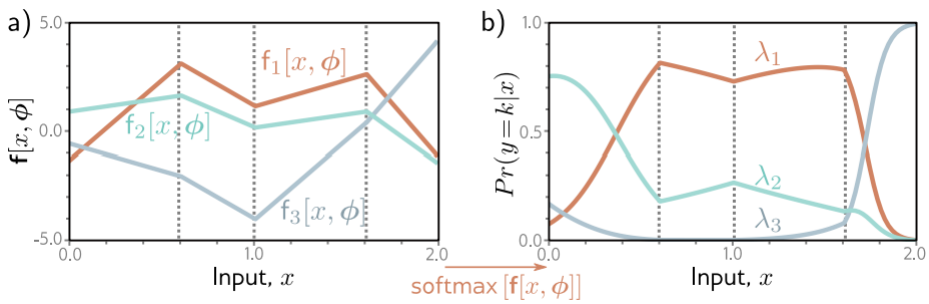
The objective of multiclass classification is to assign an input x to one of $K>2$ classes, so $y \in \{1,2,\dots,K\}$. Following the procedure of designing a loss function, we first select a probability distribution over the output space y. Since $y \in \{1,2,\dots,K\}$, a natural choice is the categorical distribution, which is defined over this domain. This distribution has K parameters $\lambda_1, \lambda_2, \dots, \lambda_K$, each representing the probability of the corresponding class. These parameters are constrained to lie between 0 and 1, and they must sum to one to form a valid probability distribution.

$$Pr(y = k|x) = \lambda_k$$



Next, we use a network $f(x, \phi)$ with K outputs to predict these K parameters from the input x. However, the raw network outputs do not automatically satisfy the constraints that each parameter lies between 0 and 1 and that they sum to one. To enforce these constraints, we apply a function that transforms the outputs accordingly. A common choice is the softmax function, which takes an arbitrary vector of length K and returns a vector of the same length with values in $[0, 1]$ that sum to one.
The kth output of the softmax function is given by:

$$\lambda_k = \text{softmax}_k[f(x, \phi)] = \frac{e^{f_k(x,\phi)}}{\sum_{j=1}^{K} e^{f_j(x,\phi)}}$$



The likelihood that an input x belongs to class $y = k$ is therefore:

$$Pr(y = k|x) = \text{softmax}_k[f(x, \phi)]$$

The corresponding loss function is the negative log-likelihood over the training dataset:

$$L[\phi] = -\sum_{i=1}^{n} log[Pr(y_i = k|f[x_i, \phi])]$$

$$L(\phi) = -\sum_{i=1}^{n} f_{y_i}(x_i, \phi) - log\left[\sum_{j=1}^{K} e^{f_j(x_i,\phi)}\right]$$

This loss is commonly called the multiclass cross-entropy loss.
After applying the softmax, the network output defines a categorical distribution over the possible classes $y \in \{1, 2, \dots, K\}$. For a single point prediction, we select the class with the highest probability:

$$\hat{y} = \arg\max_k Pr(y = k \mid f[x, \hat{\phi}])$$

which corresponds to the class with the tallest curve for that input x the above Figure.